

Chapter 3: Memory Management

Questions and Answers

Question 1

The IBM 360 had a scheme of locking 2-KB blocks by assigning each one a 4-bit key and having the CPU compare the key on every memory reference to the 4-bit key in the PSW. Name two drawbacks of this scheme not mentioned in the text.

Answer 1

First, it needs special fast hardware because the key comparison happens on every memory reference. Second, a 4-bit key allows only 16 different keys, so only a limited number of protected programs can be resident at once, including the operating system.

Question 2

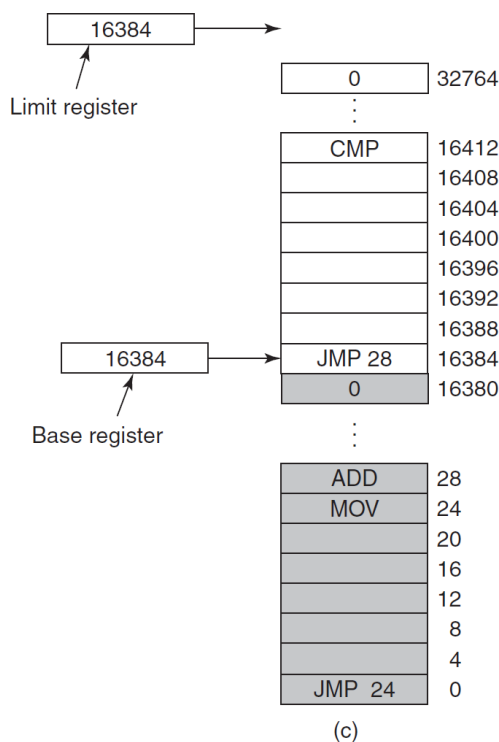


Figure 3-3. Base and limit registers can be used to give each process a separate address space.

In Fig. 3-3 the base and limit registers contain the same value, 16,384. Is this just an accident, or are they always the same? It is just an accident, why are they the same in this example?

Answer 2

No. It is only an accident in that example. The base is 16,384 because the program was loaded starting at physical address 16,384. The limit is 16,384 because the program length is 16,384 bytes.

Question 3

A swapping system eliminates holes by compaction. Assuming a random distribution of many holes and many data segments and a time to read or write a 32-bit memory word of 4 nsec, about how long does it take to compact 4 GB? For simplicity, assume that word 0 is part of a hole and that the highest word in memory contains valid data.

Answer 3

Almost all memory must be copied. A 32-bit word is 4 bytes. Reading 4 bytes takes 4 nsec, so reading 1 byte takes 1 nsec. Writing it also costs about 1 nsec, so the copy cost is about 2 nsec/byte.

$$\text{rate} = \frac{1}{2 \text{ nsec/byte}} = 500,000,000 \text{ bytes/sec}$$

$$\text{time} = \frac{2^{32}}{500,000,000} \approx 8.59 \text{ sec}$$

about 8.6 sec

Question 4

Consider a swapping system in which memory consists of the following hole sizes in memory order: 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, and 15 MB. Which hole is taken for successive segment requests of (a) 12 MB (b) 10 MB (c) 9 MB for first fit? Now repeat the question for best fit, worst fit, and next fit.

Answer 4

Algorithm	Holes selected for 12, 10, 9 MB
First fit	20 MB, 10 MB, 18 MB
Best fit	12 MB, 10 MB, 9 MB
Worst fit	20 MB, 18 MB, 15 MB
Next fit	20 MB, 18 MB, 9 MB

Question 5

What is the difference between a physical address and a virtual address?

Answer 5

A **physical address** is the actual address placed on the memory bus and understood by RAM chips. A **virtual address** is the address generated by a process inside its own address space. The MMU translates virtual addresses to physical addresses.

Question 6

For each of the following decimal virtual addresses, compute the virtual page number and offset for a 4-KB page and for an 8 KB page: 20000, 32768, 60000.

Answer 6

$$\text{page} = \left\lfloor \frac{\text{address}}{\text{page size}} \right\rfloor, \quad \text{offset} = \text{address mod page size}$$

Address	4-KB page	8-KB page
20000	(4, 3616)	(2, 3616)
32768	(8, 0)	(4, 0)
60000	(14, 2656)	(7, 2656)

Question 7

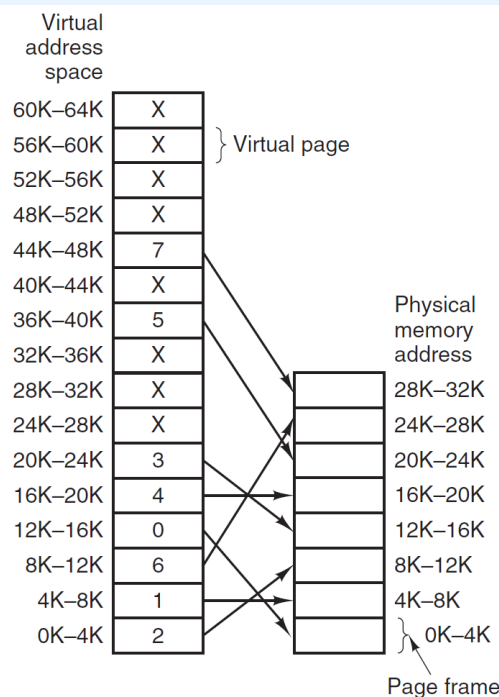


Figure 3-9. The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K–8K really means 4096–8191 and 8K to 12K means 8192–12287.

Using the page table of Fig. 3-9, give the physical address corresponding to each of the following virtual addresses: (a) 20 (b) 4100 (c) 8300

Answer 7

The physical addresses are:

20 $\mapsto$ 8212, 4100 $\mapsto$ 4100, 8300 $\mapsto$ 24684

Question 8

The Intel 8086 processor did not have an MMU or support virtual memory. Nevertheless, some companies sold systems that contained an unmodified 8086 CPU and did paging. Make an educated guess as to how they did it. (Hint: Think about the logical location of the MMU.)

Answer 8

They could place an external MMU between the CPU and the memory bus. The 8086's outgoing addresses would be treated as virtual addresses by the external MMU, which would translate them before sending physical addresses to memory.

Question 9

What kind of hardware support is needed for a paged virtual memory to work?

Answer 9

The system needs an MMU to translate virtual pages to physical frames and hardware support to trap to the operating system when a referenced page is not present or violates protection.

Question 10

Copy on write is an interesting idea used on server systems. Does it make any sense on a smartphone?

Answer 10

Yes, if the smartphone supports multiprogramming and process creation with shared pages. Modern smartphone operating systems do support multiple processes, so copy on write can reduce unnecessary copying after process creation or when shared pages are used.

Question 11

Consider the following C program:

```
int X[N];
int step = M; /* M is some predefined constant */

for (int i = 0; i < N; i += step)
    X[i] = X[i] + 1;
```

- (a) If this program is run on a machine with a 4-KB page size and 64-entry TLB, what values of M and N will cause a TLB miss for every execution of the inner loop?
- (b) Would your answer in part (a) be different if the loop were repeated many times? Explain.

Answer 11

- (a) M must make consecutive references land on different pages. With 4-KB pages, the step must be at least one page worth of array elements. If an `int` is 4 bytes:

$$M \geq \frac{4096}{4} = 1024$$

N must be large enough to generate the required sequence of page references.

- (b) Yes. If the loop is repeated many times, the referenced array footprint should exceed the TLB coverage. With 64 TLB entries and 4-KB pages:

$$64 \times 4 \text{ KB} = 256 \text{ KB}$$

So repeated passes keep missing when the referenced part of X is larger than 256 KB.

Question 12

The amount of disk space that must be available for page storage is related to the maximum number of processes, n , the number of bytes in the virtual address space, v , and the number of

bytes of RAM, r . Give an expression for the worst-case disk-space requirements. How realistic is this amount?

Answer 12

Worst-case virtual storage for all processes is nv . Since r bytes can be in RAM, the disk requirement is:

$$nv - r$$

This is usually unrealistic because systems rarely have all n processes simultaneously using their full virtual address spaces.

Question 13

If an instruction takes 1 nsec and a page fault takes an additional n nsec, give a formula for the effective instruction time if page faults occur every k instructions.

Answer 13

The fault overhead per instruction is n/k . Therefore:

$$T_{\text{effective}} = 1 + \frac{n}{k} \text{ nsec}$$

Question 14

A machine has a 32-bit address space and an 8-KB page. The page table is entirely in hardware, with one 32-bit word per entry. When a process starts, the page table is copied to the hardware from memory, at one word every 100 nsec. If each process runs for 100 msec (including the time to load the page table), what fraction of the CPU time is devoted to loading the page tables?

Answer 14

An 8-KB page has 2^{13} bytes. The number of pages is:

$$\frac{2^{32}}{2^{13}} = 2^{19} = 524,288$$

Loading time:

$$524,288 \times 100 \text{ nsec} = 52.4288 \text{ msec}$$

Fraction:

$$\frac{52.4288}{100} \approx 0.52$$

$$\boxed{\text{about } 52\%}$$

Question 15

Suppose that a machine has 48-bit virtual addresses and 32-bit physical addresses. (a) If pages are 4 KB, how many entries are in the page table if it has only a single level? Explain. (b) Suppose this same system has a TLB (Translation Lookaside Buffer) with 32 entries. Furthermore, suppose that a program contains instructions that fit into one page and it sequentially

reads long integer elements from an array that spans thousands of pages. How effective will the TLB be for this case?

Answer 15

With 4-KB pages, the offset is 12 bits, so the virtual page number has $48 - 12 = 36$ bits.

$$2^{36} \text{ page table entries}$$

For the TLB, instruction references can hit almost all the time if the code fits in one page. Sequential long-integer data access has one miss per data page. A 4-KB page contains 1024 4-byte long integers, so there is about one data TLB miss per 1024 data references.

Question 16

You are given the following data about a virtual memory system: (a) The TLB can hold 1024 entries and can be accessed in 1 clock cycle (1 nsec). (b) A page table entry can be found in 100 clock cycles or 100 nsec. (c) The average page replacement time is 6 msec. If page references are handled by the TLB 99% of the time, and only 0.01% lead to a page fault, what is the effective address-translation time?

Answer 16

Probabilities:

$$P(\text{TLB hit}) = 0.99, \quad P(\text{page-table lookup}) = 0.0099, \quad P(\text{page fault}) = 0.0001$$

$$T = 0.99(1) + 0.0099(100) + 0.0001(6 \times 10^6)$$

$$T = 0.99 + 0.99 + 600 = 601.98 \text{ nsec}$$

$$T \approx 602 \text{ nsec}$$

Question 17

Suppose that a machine has 38-bit virtual addresses and 32-bit physical addresses. (a) What is the main advantage of a multilevel page table over a single-level one? (b) With a two-level page table, 16-KB pages, and 4-byte entries, how many bits should be allocated for the top-level page table field and how many for the next-level page table field? Explain.

Answer 17

Multilevel page tables avoid keeping unused parts of a huge page table in memory. For 16-KB pages, the offset is 14 bits. The remaining virtual page bits are:

$$38 - 14 = 24$$

Each page table page holds:

$$\frac{16 \text{ KB}}{4} = 4096 = 2^{12} \text{ entries}$$

So use:

$$12 \text{ bits for the top level and 12 bits for the next level}$$

Question 18

Section 3.3.4 states that the Pentium Pro extended each entry in the page table hierarchy to 64 bits but still could only address 4 GB of memory. Explain how this statement can be true when page table entries have 64 bits.

Answer 18

The page table entry size increased to hold more control and addressing information, but the virtual address format still used only 32 bits. The format changed from roughly (10, 10, 12) to (2, 9, 9, 12), still totaling 32 bits.

Question 19

A computer with a 32-bit address uses a two-level page table. Virtual addresses are split into a 9-bit top-level page table field, an 11-bit second-level page table field, and an offset. How large are the pages and how many are there in the address space?

Answer 19

Page-number bits:

$$9 + 11 = 20$$

Offset bits:

$$32 - 20 = 12$$

Page size:

$$2^{12} = 4096 \text{ bytes} = 4 \text{ KB}$$

Number of virtual pages:

$$2^{20} \text{ pages}$$

Question 20

A computer has 32-bit virtual addresses and 4-KB pages. The program and data together fit in the lowest page (0–4095). The stack fits in the highest page. How many entries are needed in the page table if traditional (one-level) paging is used? How many page table entries are needed for two-level paging, with 10 bits in each part?

Answer 20

One-level paging needs one entry for every virtual page:

$$\frac{2^{32}}{2^{12}} = 2^{20} = 1,048,576$$

With two-level paging, only the top-level table plus the two needed second-level tables are needed. Conceptually, this means one top-level table and two active lower-level entries/tables for the bottom and top pages.

Question 21

Below is an execution trace of a program fragment for a computer with 512-byte pages. The program is located at address 1020, and its stack pointer is at 8192 (the stack grows toward 0). Give the page reference string generated by this program. Each instruction occupies 4 bytes (1 word) including immediate constants. Both instruction and data references count in the reference string.

1. Load word 6144 into register 0.
2. Push register 0 onto the stack.
3. Call a procedure at 5120, stacking the return address.
4. Subtract the immediate constant 16 from the stack pointer.
5. Compare the actual parameter to the immediate constant 4.
6. Jump if equal to 5152.

Answer 21

With 512-byte pages:

$$\text{page} = \left\lfloor \frac{\text{address}}{512} \right\rfloor$$

The reference string is:

$$1(I), 12(D), 2(I), 15(D), 2(I), 15(D), 10(I)$$

I means instruction reference and D means data reference.

Question 22

A computer whose processes have 1024 pages in their address spaces keeps its page tables in memory. The overhead required for reading a word from the page table is 5 nsec. To reduce this overhead, the computer has a TLB, which holds 32 (virtual page, physical page frame) pairs, and can do a lookup in 1 nsec. What hit rate is needed to reduce the mean overhead to 2 nsec?

Answer 22

Let h be the hit rate.

$$2 = 1h + 5(1 - h)$$

$$2 = 5 - 4h$$

$$h = 0.75$$

$$\boxed{75\% \text{ hit rate}}$$

Question 23

How can the associative memory device needed for a TLB be implemented in hardware, and what are the implications of such a design for expandability?

Answer 23

Associative memory compares the searched virtual page number against all stored virtual page numbers in parallel. Each entry needs comparators for its bits. Hardware cost grows roughly linearly with the number of entries, so making a TLB much larger becomes expensive.

Question 24

A machine has 48-bit virtual addresses and 32-bit physical addresses. Pages are 8 KB. How many entries are needed for a single-level linear page table?

Answer 24

An 8-KB page has 2^{13} bytes, so the virtual page number has:

$$48 - 13 = 35 \text{ bits}$$

$$2^{35} \text{ entries}$$

Question 25

A computer with an 8-KB page, a 256-KB main memory, and a 64-GB virtual address space uses an inverted page table to implement its virtual memory. How big should the hash table be to ensure a mean hash chain length of less than 1? Assume that the hash table size is a power of two.

Answer 25

Number of physical page frames:

$$\frac{256 \text{ KB}}{8 \text{ KB}} = 32$$

To keep mean chain length below 1, the hash table needs more slots than entries. Using a power of two, choose:

$$64 \text{ hash-table entries}$$

Question 26

A student in a compiler design course proposes to the professor a project of writing a compiler that will produce a list of page references that can be used to implement the optimal page replacement algorithm. Is this possible? Why or why not? Is there anything that could be done to improve paging efficiency at run time?

Answer 26

Generally no. The exact future reference string depends on runtime inputs, branches, data values, and system behavior. A compiler can improve locality by placing related procedures and data close together, but it cannot usually know the exact future page references.

Question 27

Suppose that the virtual page reference stream contains repetitions of long sequences of page references followed occasionally by a random page reference. For example, the sequence: 0, 1, ... , 511, 431, 0, 1, ... , 511, 332, 0, 1, ... consists of repetitions of the sequence 0, 1, ... , 511 followed by a random reference to pages 431 and 332. (a) Why will the standard replacement algorithms (LRU, FIFO, clock) not be effective in handling this workload for a page allocation that is less than the sequence length? (b) If this program were allocated 500 page frames, describe a page replacement approach that would perform much better than the LRU, FIFO, or clock algorithms.

Answer 27

If fewer than 512 frames are available, the sequential scan pushes out pages that will soon be needed again. LRU, FIFO, and clock all behave badly because the working sequence is longer than memory.

With 500 frames, a better approach is to keep pages 0 through 498 fixed and use one frame for the occasional random page. This protects most of the repeated sequence.

Question 28

If FIFO page replacement is used with four page frames and eight pages, how many page faults will occur with the reference string 0172327103 if the four frames are initially empty? Now repeat this problem for LRU.

Answer 28

FIFO page faults = 6

LRU page faults = 7

Question 29

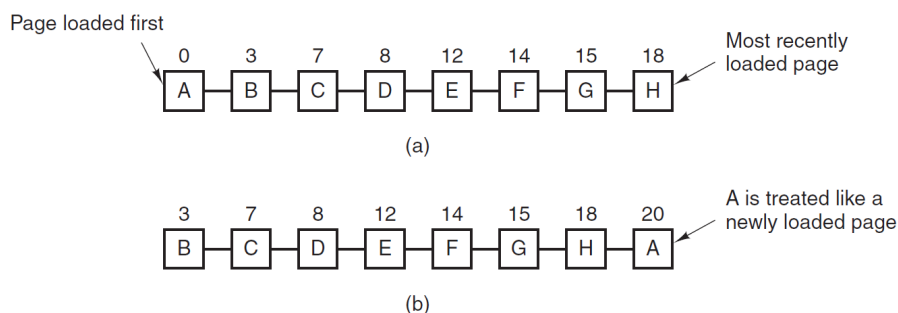


Figure 3-15. Operation of second chance. (a) Pages sorted in FIFO order. (b) Page list if a page fault occurs at time 20 and A has its *R* bit set. The numbers above the pages are their load times.

Consider the page sequence of Fig. 3-15(b). Suppose that the *R* bits for the pages B through A are 11011011, respectively. Which page will second chance remove?

Answer 29

Second chance scans from the oldest page and skips pages whose *R* bit is 1, clearing those bits. The first page encountered with *R* = 0 is removed.

Page D is removed

Question 30

A small computer on a smart card has four page frames. At the first clock tick, the R bits are 0111 (page 0 is 0, the rest are 1). At subsequent clock ticks, the values are 1011, 1010, 1101, 0010, 1010, 1100, and 0001. If the aging algorithm is used with an 8-bit counter, give the values of the four counters after the last tick.

Answer 30

After each clock tick, each counter shifts right by one bit and the current R bit is inserted at the left.

Page	Final counter
0	01101100
1	01001001
2	00110111
3	10001011

Question 31

Give a simple example of a page reference sequence where the first page selected for replacement will be different for the clock and LRU page replacement algorithms. Assume that a process is allocated three frames, and the reference string contains page numbers from the set 0, 1, 2, 3.

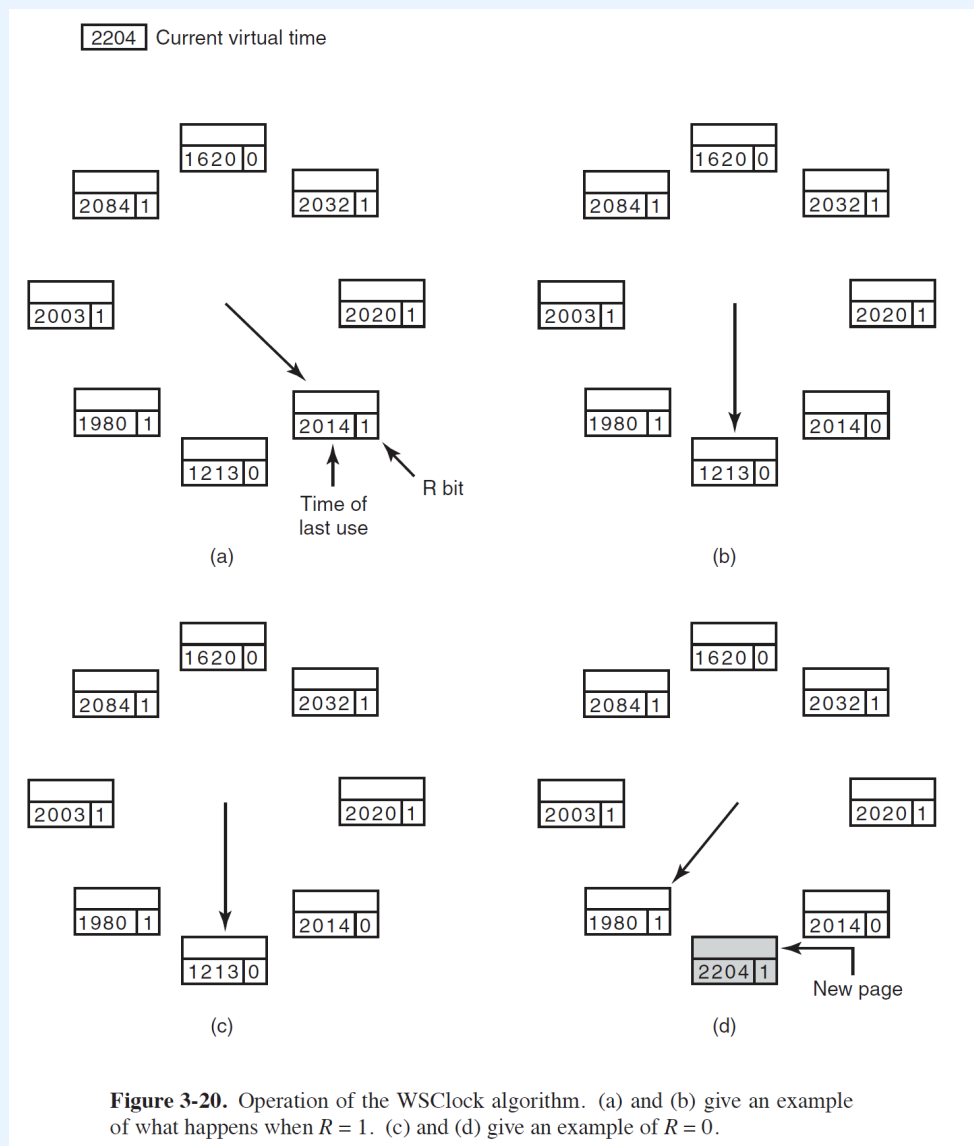
Answer 31

One example is:

0, 1, 2, 1, 2, 0, 3

When page 3 is referenced, LRU replaces page 1. Clock may replace page 0, depending on the hand position and reference bits.

Question 32



In the WSClock algorithm of Fig. 3-20(c), the hand points to a page with $R = 0$. If $\tau = 400$, will this page be removed? What about if $\tau = 1000$?

Answer 32

The age is:

$$2204 - 1213 = 991$$

If $\tau = 400$, then $991 > 400$, so the page is outside the working set and can be removed if clean. If $\tau = 1000$, then $991 \leq 1000$, so it is still inside the working set and is not removed.

Question 33

Suppose that the WSClock page replacement algorithm uses a τ of two ticks, and the system state is the following:

Page	Time stamp	V	R	M
0	6	1	0	1
1	9	1	1	0
2	9	1	1	1
3	7	1	0	0
4	4	0	0	0

where the three flag bits V, R, and M stand for Valid, Referenced, and Modified, respectively.

- (a) If a clock interrupt occurs at tick 10, show the contents of the new table entries. Explain. (You can omit entries that are unchanged.)
- (b) Suppose that instead of a clock interrupt, a page fault occurs at tick 10 due to a read request to page 4. Show the contents of the new table entries. Explain. (You can omit entries that are unchanged.)

Answer 33

- (a) On a clock interrupt at tick 10, every page with $R = 1$ gets its time stamp updated to 10, and the referenced bits are cleared. The changed entries are:

Page	Time stamp	V	R	M
1	10	1	0	0
2	10	1	0	1

Pages 0, 3, and 4 are unchanged because their R bits were already 0.

- (b) On a read fault for page 4 at tick 10, WSClock looks for a valid page with $R = 0$, age greater than τ , and $M = 0$. Page 3 satisfies this:

$$10 - 7 = 3 > 2$$

so page 3 is replaced by page 4. The changed entries are:

Page	Time stamp	V	R	M
3	7	0	0	0
4	10	1	1	0

Question 34

A student has claimed that "in the abstract, the basic page replacement algorithms (FIFO, LRU, optimal) are identical except for the attribute used for selecting the page to be replaced."

- (a) What is that attribute for the FIFO algorithm? LRU algorithm? Optimal algorithm? (b) Give the generic algorithm for these page replacement algorithms.

Answer 34

Algorithm	Victim attribute
FIFO	Earliest load time.
LRU	Oldest last-reference time.
Optimal	Farthest next-reference time in the future.

Generic algorithm: label each resident page with the relevant attribute, then evict the page with the worst label according to that algorithm.

Question 35

How long does it take to load a 64-KB program from a disk whose average seek time is 5 msec, whose rotation time is 5 msec, and whose tracks hold 1 MB (a) for a 2-KB page size? (b) for a 4-KB page size? The pages are spread randomly around the disk and the number of cylinders is so large that the chance of two pages being on the same cylinder is negligible.

Answer 35

Seek plus rotation costs 10 msec per page. Transfer time is small.

For 2-KB pages, there are $64/2 = 32$ pages. Time is about:

$$32(10.009766) \approx 320.31 \text{ msec}$$

For 4-KB pages, there are $64/4 = 16$ pages. Time is about:

$$16(10.01953) \approx 160.31 \text{ msec}$$

4-KB pages load faster here because fewer disk operations are needed.

Question 36

A computer has four page frames. The time of loading, time of last access, and the R and M bits for each page are as shown below (the times are in clock ticks):

Page	Loaded	Last ref.	R	M
0	126	280	1	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

- (a) Which page will NRU replace?
- (b) Which page will FIFO replace?
- (c) Which page will LRU replace?
- (d) Which page will second chance replace?

Answer 36

Algorithm	Victim
NRU	2
FIFO	3
LRU	1
Second chance	2

Question 37

Suppose that two processes A and B share a page that is not in memory. If process A faults on the shared page, the page table entry for process A must be updated once the page is read into memory. (a) Under what conditions should the page table update for process B be delayed even though the handling of process A's page fault will bring the shared page into memory? Explain. (b) What is the potential cost of delaying the page table update?

Answer 37

Delay *B*'s update if *B* may never access the page or may access it only after it has been evicted again. The potential cost is an extra page fault for *B* later, even though the page may already be in memory.

Question 38

Consider the following two-dimensional array:

```
int X[64][64];
```

Suppose that a system has four page frames and each frame is 128 words (an integer occupies one word). Programs that manipulate the X array fit into exactly one page and always occupy page 0. The data are swapped in and out of the other three frames. The X array is stored in row-major order (i.e., $X[0][1]$ follows $X[0][0]$ in memory). Which of the two code fragments shown below will generate the lowest number of page faults? Explain and compute the total number of page faults.

Fragment A

```
for (int j = 0; j < 64; j++)
    for (int i = 0; i < 64; i++)
        X[i][j] = 0;
```

Fragment B

```
for (int i = 0; i < 64; i++)
    for (int j = 0; j < 64; j++)
        X[i][j] = 0;
```

Answer 38

Fragment B has fewer page faults because it accesses the array in row-major order, giving good spatial locality.

Each page holds 128 integers. Each row has 64 integers, so two rows fit in one page. Fragment B faults once every two rows:

32 data page faults

Fragment A scans by columns, causing poor locality:

Question 39

You have been hired by a cloud computing company that deploys thousands of servers at each of its data centers. They have recently heard that it would be worthwhile to handle a page fault at server A by reading the page from the RAM memory of some other server rather than its local disk drive. (a) How could that be done? (b) Under what conditions would the approach be worthwhile? Be feasible?

Answer 39

The system could treat remote RAM as a network-backed swap area. On a page fault, server A requests the page from another server that holds a copy in RAM. It is worthwhile if network RAM access is much faster than local disk I/O, software overhead is low, there is enough idle remote RAM, and reliability problems are handled. Since RAM is volatile, failure of the remote server can lose the backing copy unless replication is used.

Question 40

One of the first timesharing machines, the DEC PDP-1, had a (core) memory of 4K 18-bit words. It held one process at a time in its memory. When the scheduler decided to run another process, the process in memory was written to a paging drum, with 4K 18-bit words around the circumference of the drum. The drum could start writing (or reading) at any word, rather than only at word 0. Why do you suppose this drum was chosen?

Answer 40

It avoided rotational latency. If the drum could begin at any word, the system did not have to wait for word 0 to rotate under the head before transferring a process image.

Question 41

A computer provides each process with 65,536 bytes of address space divided into pages of 4096 bytes each. A particular program has a text size of 32,768 bytes, a data size of 16,386 bytes, and a stack size of 15,870 bytes. Will this program fit in the machine's address space? Suppose that instead of 4096 bytes, the page size were 512 bytes, would it then fit? Each page must contain either text, data, or stack, not a mixture of two or three of them.

Answer 41

With 4096-byte pages:

$$\text{text} = 8, \quad \text{data} = 5, \quad \text{stack} = 4$$

Total:

$$8 + 5 + 4 = 17 \text{ pages}$$

But the address space has only $65536/4096 = 16$ pages, so it does not fit.

With 512-byte pages:

$$\text{text} = 64, \quad \text{data} = 33, \quad \text{stack} = 31$$

Total:

$$64 + 33 + 31 = 128$$

The address space has $65536/512 = 128$ pages, so it fits.

Question 42

It has been observed that the number of instructions executed between page faults is directly proportional to the number of page frames allocated to a program. If the available memory is doubled, the mean interval between page faults is also doubled. Suppose that a normal instruction takes 1 microsec, but if a page fault occurs, it takes $2001 \mu\text{sec}$ (i.e., 2 msec) to handle the fault. If a program takes 60 sec to run, during which time it gets 15,000 page faults, how long would it take to run if twice as much memory were available?

Answer 42

Each page fault adds about 2 msec of overhead:

$$15000 \times 2 \text{ msec} = 30 \text{ sec}$$

So the original run is 30 sec useful execution plus 30 sec page-fault overhead. Doubling memory halves the page faults, so overhead becomes 15 sec.

$$30 + 15 = 45 \text{ sec}$$

Question 43

A group of operating system designers for the Frugal Computer Company are thinking about ways to reduce the amount of backing store needed in their new operating system. The head guru has just suggested not bothering to save the program text in the swap area at all, but just page it in directly from the binary file whenever it is needed. Under what conditions, if any, does this idea work for the program text? Under what conditions, if any, does it work for the data?

Answer 43

Yes for program text if it is read-only. If evicted, it can be reread from the executable file. This usually does not work for data, because data pages are modified during execution. Writing modified data back into the executable would corrupt the program's initial data for future runs.

Question 44

A machine-language instruction to load a 32-bit word into a register contains the 32-bit address of the word to be loaded. What is the maximum number of page faults this instruction can cause?

Answer 44

The instruction itself can straddle a page boundary, causing two faults while fetching the instruction. The 32-bit word being loaded can also straddle a page boundary, causing two more faults.

$$4 \text{ page faults}$$

If the machine requires aligned words, the data word may cause only one page fault, reducing the maximum.

Question 45

Explain the difference between internal fragmentation and external fragmentation. Which one occurs in paging systems? Which one occurs in systems using pure segmentation?

Answer 45

Internal fragmentation is wasted space inside an allocated unit, such as the unused part of the last page. **External fragmentation** is wasted space between allocated units.

Paging suffers mainly from internal fragmentation. Pure segmentation suffers mainly from external fragmentation.

Question 46

When segmentation and paging are both being used, as in MULTICS, first the segment descriptor must be looked up, then the page descriptor. Does the TLB also work this way, with two levels of lookup?

Answer 46

No. The TLB lookup key can include both the segment number and the virtual page number. This lets the TLB find the final page mapping in one associative match.

Question 47

We consider a program which has the two segments shown below consisting of instructions in segment 0, and read/write data in segment 1. Segment 0 has read/execute protection, and segment 1 has just read/write protection. The memory system is a demand-paged virtual memory system with virtual addresses that have a 4-bit page number, and a 10-bit offset. The page tables and protection are as follows (all numbers in the table are in decimal):

Segment 0: Read/Execute		Segment 1: Read/Write	
Virtual page #	Page frame #	Virtual page #	Page frame #
0	2	0	On Disk
1	On Disk	1	14
2	1	2	12
3	536	3	9
4	On Disk	4	On Disk
5	On Disk	5	13
6	468	6	73
7	1	7	2

For each of the following cases, either give the real (actual) memory address which results from dynamic address translation or identify the type of fault which occurs (either page or protection fault).

- (a) Fetch from segment 1, page 1, offset 3.
- (b) Store into segment 0, page 0, offset 16.
- (c) Fetch from segment 1, page 4, offset 28.
- (d) Jump to location in segment 1, page 3, offset 32.

Answer 47

Case	Result
(a)	Real address (14, 3), equivalently physical frame 14 with offset 3.
(b)	Protection fault: writing to read/execute segment 0.
(c)	Page fault: segment 1, page 4 is on disk.
(d)	Protection fault: jumping to read/write data segment 1.

Question 48

Can you think of any situations where supporting virtual memory would be a bad idea, and what would be gained by not having to support virtual memory? Explain.

Answer 48

Virtual memory may be unnecessary in small embedded systems, smart cards, or special-purpose processors where memory requirements are fixed and controlled. Omitting it simplifies the OS, reduces code size, removes page-fault overhead, and avoids MMU complexity.

Question 49

Virtual memory provides a mechanism for isolating one process from another. What memory management difficulties would be involved in allowing two operating systems to run concurrently? How might these difficulties be addressed?

Answer 49

This is a virtualization problem. Each OS expects to control virtual memory and the TLB. The difficulty is switching memory contexts quickly while isolating each OS. Solutions include software-handled TLB misses, shadow or nested page tables, and TLB entries tagged with context identifiers so mappings from different operating systems do not conflict.

Question 50

Plot a histogram and calculate the mean and median of the sizes of executable binary files on a computer to which you have access. On a Windows system, look at all .exe and .dll files; on a UNIX system look at all executable files in /bin, /usr/bin, and /local/bin that are not scripts (or use the file utility to find all executables). Determine the optimal page size for this computer just considering the code (not data). Consider internal fragmentation and page table size, making some reasonable assumption about the size of a page table entry. Assume that all programs are equally likely to be run and thus should be weighted equally.

Answer 50

Collect executable file sizes, plot their histogram, then compute:

$$\text{mean} = \frac{\sum_i s_i}{n}$$

and the median as the middle sorted size. For a process size s and page-table entry size e , the simplified page-size tradeoff is:

$$\text{overhead} = \frac{se}{p} + \frac{p}{2}$$

The approximate optimum is:

$$p = \sqrt{2se}$$

Use the measured mean executable size as s , choose a reasonable e such as 4 or 8 bytes, then round p to a supported page size.

Question 51

Write a program that simulates a paging system using the aging algorithm. The number of page frames is a parameter. The sequence of page references should be read from a file. For a given input file, plot the number of page faults per 1000 memory references as a function of the number of page frames available.

Answer 51

Use one counter per resident page. On each clock tick:

1. shift every counter right by 1;
2. insert the current referenced bit at the left;
3. clear the referenced bits.

On a page fault, evict the page with the smallest counter. Run the same reference file for different frame counts and record:

$$\text{fault rate} = \frac{\text{faults}}{\text{references}} \times 1000$$

Question 52

Write a program that simulates a toy paging system that uses the WSClock algorithm. The system is a toy in that we will assume there are no write references (not very realistic), and process termination and creation are ignored (eternal life).

The inputs will be:

- The reclamation age threshold.
- The clock interrupt interval expressed as number of memory references.
- A file containing the sequence of page references.

- (a) Describe the basic data structures and algorithms in your implementation.
- (b) Show that your simulation behaves as expected for a simple (but nontrivial) input example.
- (c) Plot the number of page faults and working set size per 1000 memory references.
- (d) Explain what is needed to extend the program to handle a page reference stream that also includes writes.

Answer 52

Use a circular list of frames. Each frame stores page number, valid bit, referenced bit, modified bit, and last-use time. For the toy read-only version, $M = 0$ always.

On a reference, set $R = 1$. On page fault, advance the clock hand until it finds a page with $R = 0$ and age greater than τ . If $R = 1$, clear it and update last-use time.

Validate with a small reference string where the working set is known. Plot page faults and

working-set size per 1000 references. To support writes, include write references, set $M = 1$ on writes, and schedule dirty pages for write-back before eviction.

Question 53

Write a program that demonstrates the effect of TLB misses on the effective memory access time by measuring the per-access time it takes to stride through a large array.

- (a) Explain the main concepts behind the program, and describe what you expect the output to show for some practical virtual memory architecture.
- (b) Run the program on some computer and explain how well the data fit your expectations.
- (c) Repeat part (b) but for an older computer with a different architecture and explain any major differences in the output.

Answer 53

Allocate an array much larger than the TLB coverage. Access it repeatedly using different strides. Small strides should show low per-access time due to spatial locality and TLB reuse. Strides near or above the page size should increase per-access time because each access may touch a different page and cause more TLB misses.

For comparison across machines, keep the same benchmark structure but interpret differences using page size, TLB size, cache hierarchy, and prefetching behavior.

Question 54

Write a program that will demonstrate the difference between using a local page replacement policy and a global one for the simple case of two processes. You will need a routine that can generate a page reference string based on a statistical model. This model has N states numbered from 0 to $N - 1$ representing each of the possible page references and a probability p_i associated with each state i representing the chance that the next reference is to the same page. Otherwise, the next page reference will be one of the other pages with equal probability.

- (a) Demonstrate that the page reference string-generation routine behaves properly for some small N .
- (b) Compute the page fault rate for a small example in which there is one process and a fixed number of page frames. Explain why the behavior is correct.
- (c) Repeat part (b) with two processes with independent page reference sequences and twice as many page frames as in part (b).
- (d) Repeat part (c) but using a global policy instead of a local one. Also, contrast the per-process page fault rate with that of the local policy approach.

Answer 54

Generate references with N states, where state i repeats with probability p_i ; otherwise choose another page uniformly. For local replacement, each process owns a fixed number of frames. For global replacement, both processes share one frame pool.

Measure per-process and total page-fault rates. Local replacement gives isolation but may waste frames. Global replacement adapts frame allocation but one process can hurt another's fault rate.

Question 55

Write a program that can be used to compare the effectiveness of adding a tag field to TLB entries when control is toggled between two programs. The tag field is used to effectively label each entry with the process id. Note that a nontagged TLB can be simulated by requiring that all TLB entries have the same tag at any one time.

The inputs will be:

- The number of TLB entries available.
 - The clock interrupt interval expressed as number of memory references.
 - A file containing a sequence of (process, page references) entries.
 - The cost to update one TLB entry.
- (a) Describe the basic data structures and algorithms in your implementation.
- (b) Show that your simulation behaves as expected for a simple (but nontrivial) input example.
- (c) Plot the number of TLB updates per 1000 references.

Answer 55

Simulate a TLB with entries containing process id, virtual page, frame, and validity. With tags, entries for both processes can coexist. Without tags, every context switch effectively invalidates or rewrites the TLB.

For each reference, check for a matching $(pid, page)$ entry. Count TLB hits, misses, and updates. Plot TLB updates per 1000 references. Tagged TLBs should show fewer updates when processes alternate frequently and their working sets fit in the TLB.